DATABASE MANAGEMENT SYSTEMS LAB

(CSC2131)

PROJECT REPORT

Academic Session: 2026-2027

VEHICLE RENTAL MANAGEMENT SYSTEM

By:

Name of Student: Udith Praveen

Registration Number: 2502052183

Section: B.tech/O4

MANIPAL UNIVERSITY
JAIPUR

November 2026

Department of Computer Science and Engineering

Manipal University Jaipur

VPO. Dehmi Kalan, Jaipur, Rajasthan, India – 303007

DECLARATION BY THE STUDENT

I hereby declare that this project **Vehicle Rental Management System** is my own work and that, to the best of my knowledge and belief, it contains no material previously published or written by another person nor material which has been accepted for the award of any other degree or diploma of the University or other Institute, except where due acknowledgements has been made in the text.

(Signature)

Place: Jaipur

Date:

Name of Student: Udith Praveen

Registration Number: 2502052183

B.Tech CSE 3rd Semester

Section: B.tech/O4

TABLE OF CONTENTS

S. No	Contents	Page No.
1	Abstract	4
2	Introduction	4
3	Problem Statement	4
4	Objectives of the Project	4
5	System Requirements (Hardware and Software)	5
6	System Design and Database Schema	5
7	ER Diagram	5
8	Database and Table Creation	6
9	SQL Queries for Data Insertion, Updation, Deletion, and Retrieval	6
10	Front-End Design and Implementation (Case 1 only)	7
11	Results and Screenshots	7
12	Conclusion	8
13	Contributions by Team Members	8
14	References	8

1. Abstract

The Vehicle Rental Management System is a database-driven application developed for managing car rental operations. The system replaces manual paper logs by connecting a relational MySQL database with a clean Python terminal interface. It maintains information on fleet vehicles, registered customers, rental transactions, billing payments, and maintenance logs. By enforcing Primary and Foreign Key constraints, the database ensures referential integrity across all entities, while multi-table SQL INNER JOIN queries combine related records into readable rental summaries.

2. Introduction

Vehicle rental services operate in a fast-paced environment where cars are rented, returned, and serviced continuously. A reliable database system is necessary to keep track of which cars are available, who is currently renting a car, and whether maintenance has been completed. This project implements a relational database solution in MySQL integrated with a Python console front-end to streamline day-to-day rental tasks.

3. Problem Statement

When vehicle rental records are managed using paper logs or basic spreadsheets, businesses frequently run into common issues: duplicate vehicle bookings, misplaced customer driving license numbers, unrecorded return dates, and incorrect billing calculations. Without referential constraints, records become disconnected and difficult to audit.

4. Objectives of the Project

- **Database Design:** Create a normalized relational schema with 5 dedicated tables in MySQL.
- **Referential Integrity:** Link tables using Primary and Foreign Keys with cascading actions.
- **CRUD Operations:** Support adding vehicles/customers, updating car statuses, recording rentals, and billing.
- **Data Retrieval:** Use SQL INNER JOIN queries to display comprehensive rental contracts in the terminal.
- **Front-End Integration:** Provide an easy-to-use Python menu interface for database administration.

5. System Requirements (Hardware and Software)

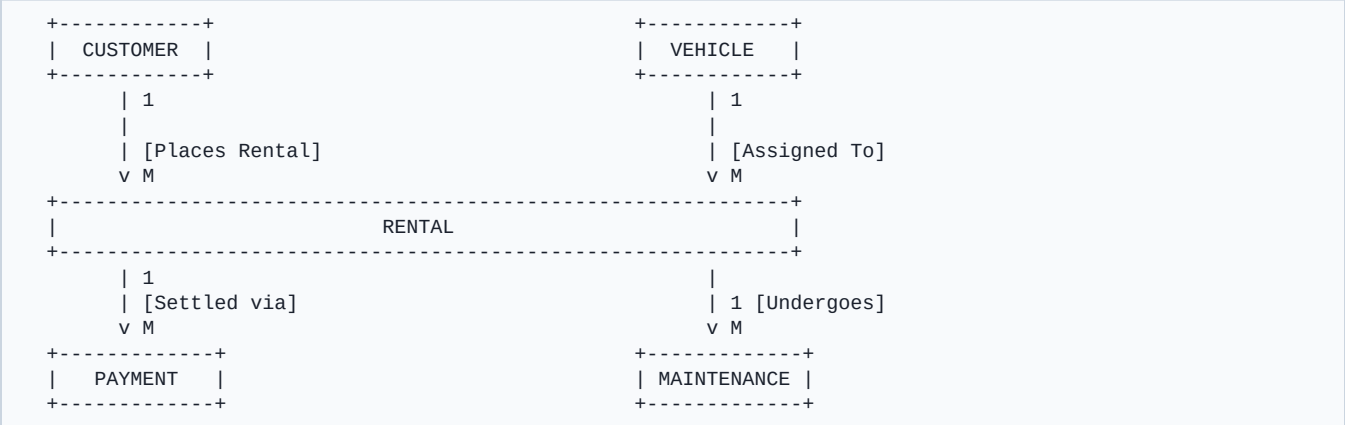
Component	Specification
Operating System	Windows 10 / 11, Linux, or macOS
Database Management System	MySQL Server 8.0+ (InnoDB Storage Engine)
Programming Language	Python 3.8 or higher
Python Database Driver	mysql-connector-python (v8.0+)
Hardware Requirements	Standard PC / Laptop (Intel Core i3 / AMD Ryzen 3+, 4 GB RAM, 500 MB Free Disk)

6. System Design and Database Schema

The system is organized into 5 relational tables:

Table	Primary Key	Foreign Key	Description & Attributes
Customer	`CustomerID`	None	FirstName, LastName, LicenseNumber (Unique), PhoneNumber
Vehicle	`VehicleID`	None	Make, Model, LicensePlate (Unique), DailyRate, Status
Rental	`RentalID`	CustomerID, VehicleID	RentalDate, ReturnDate, TotalCost. Connects customer to vehicle.
Payment	`PaymentID`	RentalID	PaymentDate, Amount, PaymentMethod (Cash/Card/UPI)
Maintenance	`MaintenanceID`	VehicleID	ServiceDate, Description, Cost. Logs vehicle servicing.

7. ER Diagram (Entity-Relationship Representation)



8. Database and Table Creation (DDL)

```
CREATE DATABASE IF NOT EXISTS rental_db;
USE rental_db;

CREATE TABLE Customer (
    CustomerID INT PRIMARY KEY,
    FirstName VARCHAR(50) NOT NULL,
    LastName VARCHAR(50) NOT NULL,
    LicenseNumber VARCHAR(30) UNIQUE NOT NULL,
    PhoneNumber VARCHAR(15) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE Vehicle (
    VehicleID INT PRIMARY KEY,
    Make VARCHAR(50) NOT NULL, Model VARCHAR(50) NOT NULL,
    LicensePlate VARCHAR(20) UNIQUE NOT NULL,
    DailyRate DECIMAL(8,2) NOT NULL, Status VARCHAR(20) DEFAULT 'Available'
) ENGINE=InnoDB;

CREATE TABLE Rental (
    RentalID INT PRIMARY KEY,
    CustomerID INT NOT NULL, VehicleID INT NOT NULL,
    RentalDate DATE NOT NULL, ReturnDate DATE, TotalCost DECIMAL(10,2) DEFAULT 0.00,
    FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID) ON DELETE CASCADE,
    FOREIGN KEY (VehicleID) REFERENCES Vehicle(VehicleID) ON DELETE CASCADE
) ENGINE=InnoDB;
```

9. SQL Queries for Data Insertion, Updation, Deletion, and Retrieval

A. Data Insertion & Status Synchronization (DML):

```
-- Renting out a vehicle (adds rental row and updates vehicle status)
INSERT INTO Rental (RentalID, CustomerID, VehicleID, RentalDate, TotalCost)
VALUES (501, 1, 101, '2026-10-25', 0.00);
UPDATE Vehicle SET Status = 'Rented' WHERE VehicleID = 101;

-- Returning a vehicle (updates return date, total cost, and restores status)
UPDATE Rental SET ReturnDate = '2026-10-28', TotalCost = 135.00 WHERE RentalID = 501;
UPDATE Vehicle SET Status = 'Available' WHERE VehicleID = 101;
```

B. Multi-Table Relational Data Retrieval (DQL - INNER JOIN):

```
SELECT
    r.RentalID,
    CONCAT(c.FirstName, ' ', c.LastName) AS Customer,
    CONCAT(v.Make, ' ', v.Model) AS Vehicle,
    v.LicensePlate, r.RentalDate,
    IFNULL(r.ReturnDate, 'Pending') AS ReturnDate, r.TotalCost
FROM Rental r
JOIN Customer c ON r.CustomerID = c.CustomerID
JOIN Vehicle v ON r.VehicleID = v.VehicleID
ORDER BY r.RentalID DESC;
```

10. Front-End Design and Implementation (Case 1 only)

The front-end is written in Python and connects directly to MySQL using mysql.connector. The project is structured into modular scripts:

- **db_config.py:** Manages database host, port, user, and password credentials. Automatically creates rental_db on startup and runs table creation queries.
- **main.py:** Implements the console menu loop, user input prompts, business validation (e.g. checking if a car is available before renting), and formatted table printing.
- **connection.py:** Entry point delegating directly to main.py.

11. Results and Screenshots

The following console outputs demonstrate the interactive menu and formatted tabular data displayed to the user:

```
------------------------------------------ | Formatted Query Output (Available Vehicles):
      VEHICLE RENTAL MANAGEMENT SYSTEM     | +------------+---------+---------+---------------+------------+-----------+
------------------------------------------ | | Vehicle ID | Make    | Model   | License Plate | Daily Rate | Status    |
 1. Add Vehicle        8. Rent a Vehicle   | +------------+---------+---------+---------------+------------+-----------+
 2. Show All Vehicles  9. Return a Vehicle | | 102        | Honda   | City    | KA-05-AB-5678 | 50.00      | Available |
 3. Available Vehicles 10. Show All Rentals| | 103        | Hyundai | Creta   | MH-02-CD-9999 | 65.00      | Available |
 4. Remove Vehicle     11. Log Payment     | +------------+---------+---------+---------------+------------+-----------+
 5. Add Customer       12. Show Payments   |
 6. Show All Customers 13. Log Maintenance | Sample Multi-Table JOIN Output (Option 10):
 7. Remove Customer    14. Show Maintenance| +-----------+-------------+--------------+---------------+------------+
 15. Setup DB Tables   0. Exit Application | | Rental ID | Customer    | Vehicle      | Rental Date   | Total Cost |
------------------------------------------ | +-----------+-------------+--------------+---------------+------------+
                                           | | 501       | Rahul Sharma | Toyota Corolla| 2026-10-25    | $0.00      |
                                           | +-----------+-------------+--------------+---------------+------------+
```

12. Conclusion

The Vehicle Rental Management System successfully demonstrates the integration of a Python terminal front-end with a MySQL relational database. The project fulfills all core requirements: database and table creation, referential integrity enforcement through Primary and Foreign Keys, CRUD data management, and data retrieval using multi-table SQL INNER JOIN queries. The modular codebase allows easy extension, such as adding a graphical user interface or online booking capabilities in the future.

13. Contributions by Team Members

This project was developed individually by **Udith Praveen** (Registration Number: **2502052183**). Key contributions include:

- Designed the 5-table relational schema and foreign key constraints in MySQL.
- Wrote the complete DDL and DML scripts in schema.sql.
- Implemented database connection handling and auto-setup in db_config.py.
- Developed the terminal menu, ASCII table renderer, and CRUD operations in main.py.
- Authored multi-table SQL INNER JOIN queries and prepared the formal documentation.

14. References

1. R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed., Pearson, 2016.
2. A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed., McGraw-Hill, 2019.
3. MySQL 8.0 Reference Manual, Oracle Corporation. URL: https://dev.mysql.com/doc/refman/8.0/en/
4. Python MySQL Connector Documentation. URL: https://dev.mysql.com/doc/connector-python/en/